Making a pinball game for Playdate: Part 07, the debugger

Tuesday, December 10, 2024

Source: https://amano.games/devlog/making-a-pinball-game-for-the-playdate-part-07-the-debugger

Welcome to this December adventure, where I write about the process of our latest game, Devils on the Moon pinball.

One of the few good programming classes I had at university was deceptively called C++. I say deceptively because we learned little about the language features, and instead the class focused heavily on the control flow of a program.

The teacher showed us how to use the Visual Studio debugger and go step by step through the code. Each class, he would teach us a new concept and walk us through it. We would go step by step in the program and showed us how to inspect the state of the program.

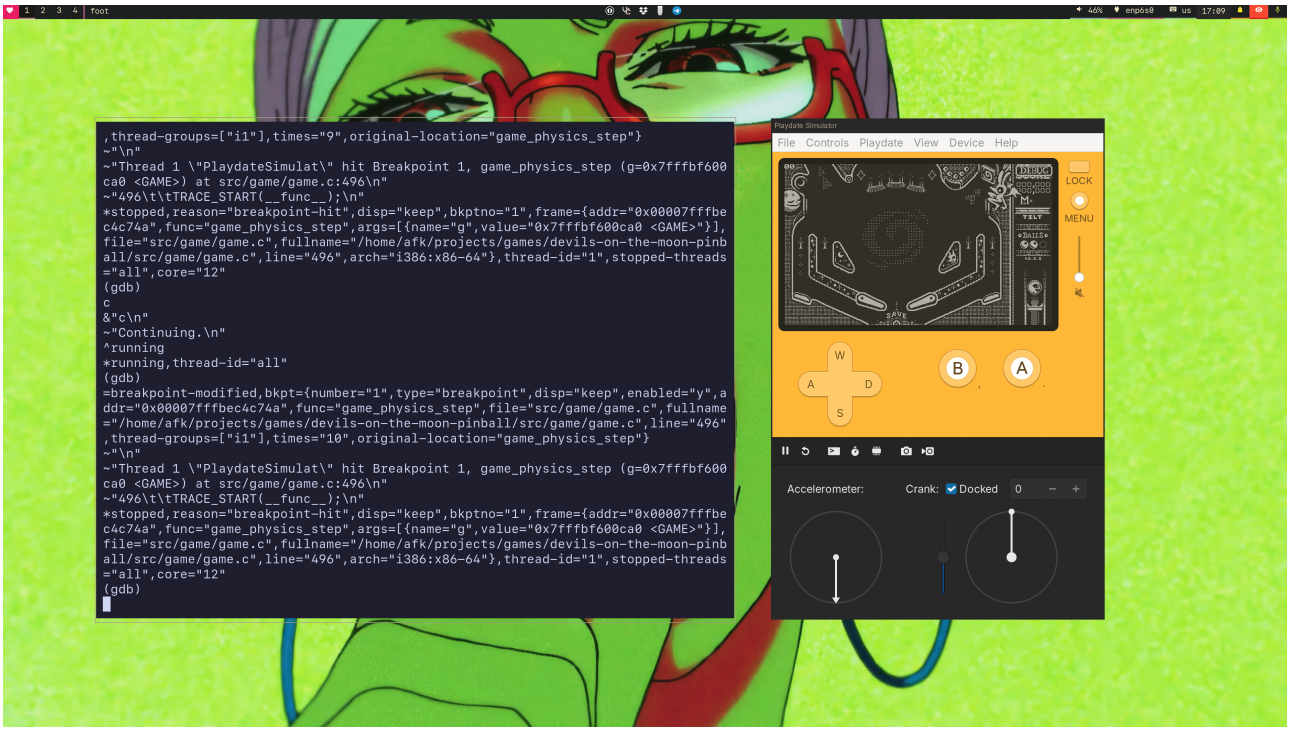
After that class, I practically stopped using a debugger even though I learned a lot using them. I eventually ended up doing `print` statements and got used to this way of working.

I used a debugger a couple of times but only for really specific issues and the majority of the time it seemed like too much hassle that wasn't worth the effort.

When I started learning C watching the Hand Made Hero video series, Casey always ran his program through the debugger. It didn't matter if he was searching for a bug or not. He dedicates a couple of hours at the beginning to setting everything up, but by the time he is actually working on his code, running the game through a debugger was as easy as running the game without it.

I also watched his video talking about Remedy BG. It certainly looked useful, and as I considered myself a novice in C I needed all the help I could get. This is how my search for a debugger that worked on Linux started.

GDB



GDB Is the de facto debugger people use on Linux. Not only that, but by the end of this adventure, I realized almost all the other options were just GDB in disguise, or a GDB front end, as they would say in their description.

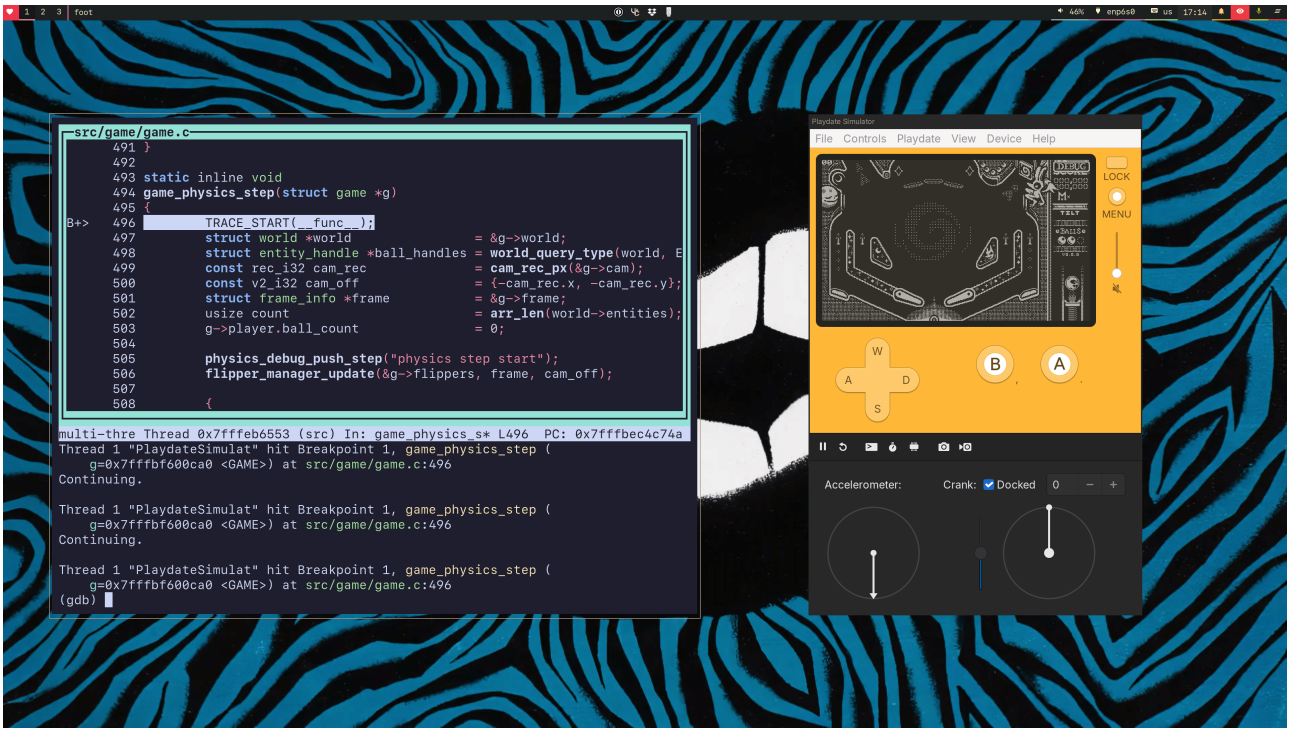
I'm not a person who shy away from the terminal and rely on tools like nvim, tmux, bat, ripgrep, lazy git, and others almost daily. However, for some reason, just like with the profiler, I was having a hard time making sense of the debugger on the CLI.

I saw a couple of tutorials, read the manual a little and learned enough to be able to use it. But by the end of the day, it never felt faster or easier than just printing stuff to the terminal.

It was really valuable though, in the future I already knew for example how to print a variable in hex or binary format, and that knowledge carried over to the other tools I tried.

So even if you don't like CLI, I would say it's valuable to learn how to use GDB. At the end of the day, a lot of tools are based on it, so you probably would end up learning how to use it even if you don't want to.

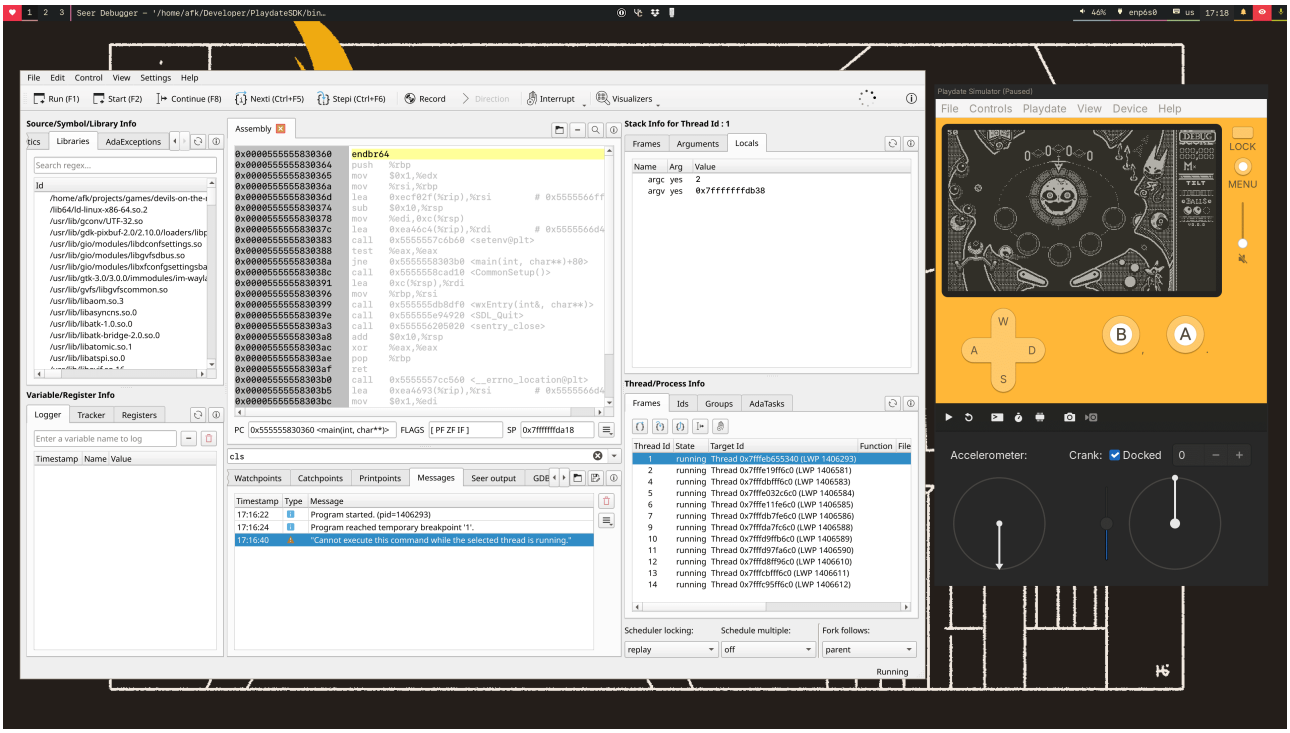
GDB TUI



The GDB TUI seems to be recommended a lot when people complain about using the CLI. It does improve things a little bit but, but not by that much. The UI somehow would always end up somewhat broken after a couple of minutes poking around at a program, and still a lot of functionality was behind the CLI interface, so I needed to learn all the syntax and small tricks that the GDB CLI interface used.

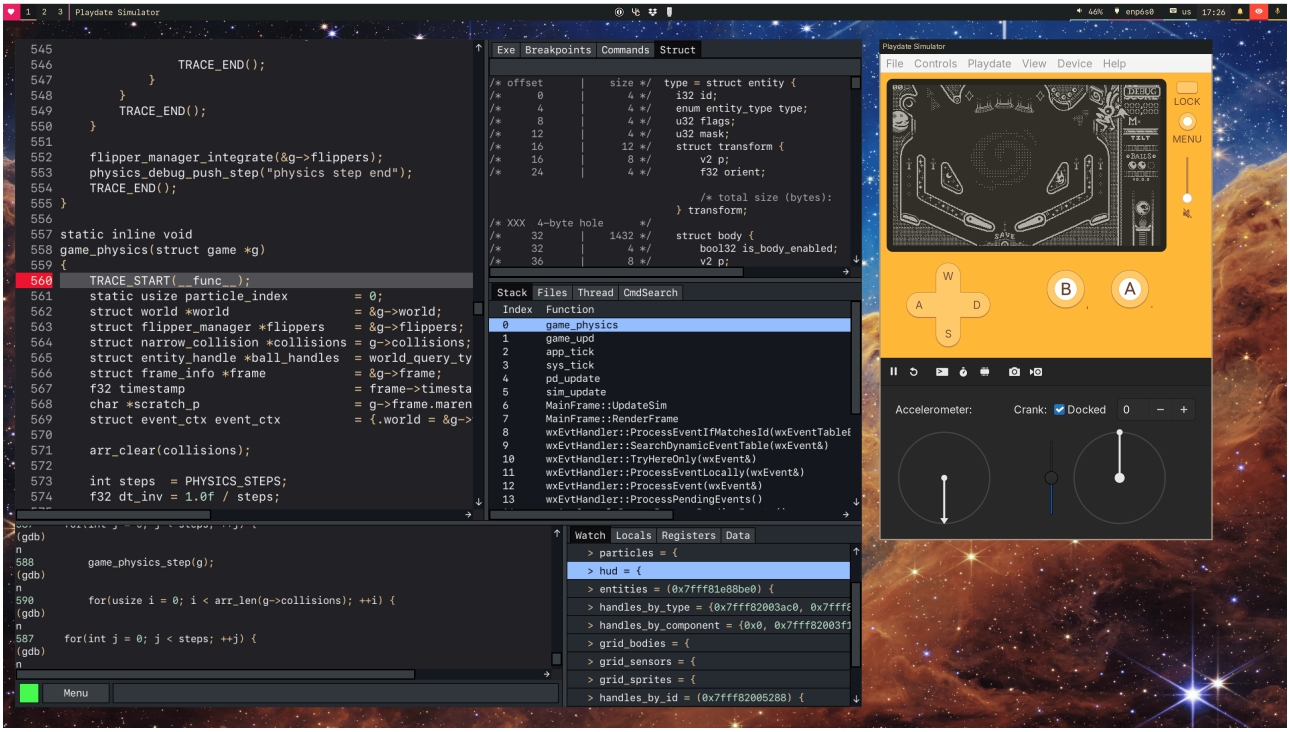
And as with all the other GDB front ends, I eventually would find something I wanted to do that GDB supported, but the TUI didn't, GDB has an enormous set of features, and it seems really hard that a GUI has support for everything that GDB can do.

Seer



Seer was one of my first tries at a GUI debugger on Linux; it sounded promising, but after using it for a day, it lacked a bunch of features, and the workflow was just too slow for me. It didn't fulfil the requirement of being more convenient than a print statement.

GF

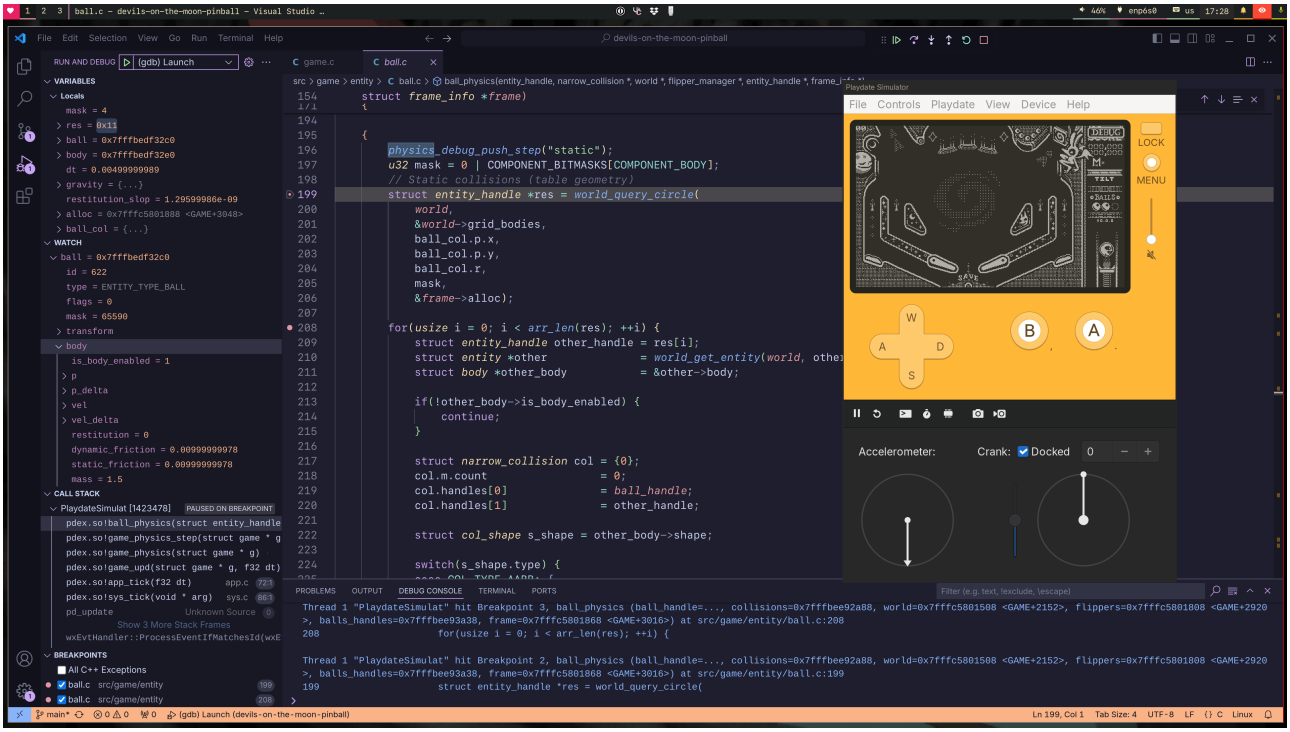


This is one of the best if not the best GDB front end, it supports a bunch of features from GDB, it works well while launching from a terminal.

One problem is that it doesn't support Wayland natively, so my code looks all blurry. Also, it has a layout system that you have to define from a config file, and having to make a change, to restart GF and then wanted to change something else quickly became tedious.

When I started using it, there was a Memory Viewer module behind a Patreon, so I subscribed to the Patreon and compiled the app with the Memory Viewer enabled. This was nice, but I would have preferred to install it directly from my package manager. After a couple of weeks, the Patreon was closed and the memory viewer plugin realized. I haven't checked in a while, but it wasn't updated in Arch yet.

VS Code C/C++ Tools



This is the option that I'm currently using. Watching Casey video on Remedy BG made me realize that a debugger could be a lot nicer than what there is right now in the Linux ecosystem, but at least with VS Code it runs fast enough, I'm already familiar with the UI, and it does most of what I need.

I have a long list of things I would like in the VS Code debugger, like being able to launch from the CLI the project attached to the debugger or a way to show my custom strings as text in the watch window. But it doesn't seem like it's going to be possible any time soon.

So that's it! I'm really excited to try out the Raddebugger once it's available on Linux. Who knows, maybe next year is actually going to be the year of the Linux desktop!

Any way, It was time to polish the physics of the game and after using this tools I had an idea.

See you in the next adventure: The physics debugger.